

IntentFuzz: A Protocol-Aware Fuzzer for Automated Invariant Violation Detection in Intent-Based Cross-Chain Bridges

André Augusto
INESC-ID & IST, University of Lisbon

André Vasconcelos
INESC-ID & IST, University of Lisbon

Christof Ferreira Torres
INESC-ID & IST, University of Lisbon

Miguel Correia
INESC-ID & IST, University of Lisbon

Abstract

Cross-chain bridges move value between blockchains. Intent-based bridges are a variant where a solver fulfills a user’s declared outcome and an off-chain settlement layer later reconciles the fill against the deposit. Existing smart-contract fuzzers and static analyzers only flag known-bad code patterns or require protocol-specific hand-written assertions. This work formalizes a taxonomy separating invariant violations, safety properties a contract must enforce locally, from settlement exposures legitimately delegated to the off-chain settlement layer, and proposes **IntentFuzz: a protocol-aware fuzzer** that recovers a bridge’s intent structure and deposit/fill function roles directly from unannotated Solidity source, then synthesizes multi-step fuzz sequences using an LLM-based fallback to help build call arguments. IntentFuzz recovers the correct intent structure in 9/9 benchmark protocols and classifies deposit and fill functions with 100% recall and 82% combined precision; across a corpus of 77 manually labeled contracts, it reaches 79.5% bridge-classification precision and 97.2% recall, and among confirmed bridges, struct selection reaches 88.6% precision and recall while deposit and fill classification each reach 100% recall. On 23 planted-bug mutants, IntentFuzz attains 100% recall and 100% precision, executing 273 templates (507 transactions in a median of 14ms per template). Across 24 real-world deployments, **it confirms 17 genuine invariant violations** under heuristic-only input generation, **rising to 22 with its LLM-assisted tier enabled**, spanning eight vulnerable GitHub repositories, each finding reproducible against public, deployed bytecode.

1 Introduction

Intent-based bridges are cross-chain protocols where users specify the outcome they want, and third-party *solvers* compete to fulfill that *intent* by executing the necessary actions across chains: for example, a user on Ethereum who wants 1,000 USDC delivered to an address on Arbitrum locks 1,000 USDC in a *deposit* describing that outcome, and a solver

fronts the funds on Arbitrum before later claiming reimbursement from the deposit (Section 2 details this deposit/solver/-fill/settlement flow) [6]. This solver-driven design changes the security model compared to normal bridges [7]. Cross-chain bridges collectively secure billions of dollars in locked value and have been the target of some of the largest exploits in decentralized finance, with total losses exceeding \$3.2 billion [2, 22, 35, 38]. This makes the correctness of any new architecture these protocols adopt a high-stakes question. ERC-7683 [16], recently proposed as a standard interface for intent-based bridges, reflects this architecture’s growing adoption, a trend this work’s own corpus of GitHub repositories independently corroborates (Section 5.1).

Existing smart contract fuzzers (rooted in the tradition of randomized testing [25]) are effective at finding bytecode or source code defects inside a single contract execution [37], but they fall short for intent-based bridges. Intent-based bridges are instead defined by cross-chain semantics, off-chain solver behavior, and multi-blockchain flows. In this setting, the space of possible failures extends beyond malformed inputs or malicious instruction sequences: it includes replay across chains, partial or delayed settlement, and other protocol-specific correctness properties that only emerge once multiple chains and an off-chain actor are considered together. Traditional fuzzers typically treat a test as a transaction or short on-chain sequence, and their assessments are usually tied to coverage, assertions, or generic vulnerability patterns [19, 33]. As a result, they cannot reliably infer the intended bridge semantics for any of these failure modes.

The security properties of intent-based bridges are qualitatively different from those of conventional smart contracts. A bridge is correct if and only if the assets received by the beneficiary on the destination chain match the intent declared on the source chain: same token, same amount, same recipient, within the declared deadline, and non-replayable. We propose a two-tier finding taxonomy. *Invariant violations* occur where the on-chain contract fails to enforce a safety property regardless of solver behavior (e.g., the same intent processed twice or after the deadline has passed). *Settlement exposures*,

by contrast, occur where the on-chain contract never checks a correctness property at all, such as whether the recipient, token, or amount a fill actually delivers matches what the deposit declared, because the protocol relies entirely on its off-chain settlement layer to catch a mismatch after the fact; if that settlement layer is itself compromised or buggy, the mismatch goes uncaught.

We present IntentFuzz, a protocol-aware fuzzer for intent-based bridges. Given an unannotated Solidity implementation of a bridge, IntentFuzz uses static analysis to extract the data structure (i.e., Solidity `struct`) that represents an intent, identify its correctness properties based on the fields present, and classify protocol functions by semantic role (e.g., deposit, fulfillment). It then constructs multi-step fuzz templates that pair source-chain deposits with destination-chain fulfillments. To handle the diversity of intent protocols, IntentFuzz infers protocol structure and safety properties automatically from source code, supporting both single-contract and cross-contract deployments. It then generates one fuzz suite per deposit-fulfillment function pair identified, so that all protocol-level entry points are covered. The evaluation of IntentFuzz is based on 1) 4 manually written reference contracts with no planted vulnerabilities (*secure baselines*), each paired with a set of mutants in which exactly one invariant check has been deliberately removed, exercising every dynamically-tested invariant in the taxonomy; 2) a corpus-scale analysis of intent-based bridge contracts found in the wild via a GitHub crawler; and 3) 24 real-world protocol/chain deployments reached through forked mainnet state.

Contributions. This paper makes the following contributions:

- **An invariant violation taxonomy for intent-based bridges.** We formalize the invariant-violation/settlement-exposure distinction introduced above into a taxonomy of six invariants that intent-based bridge contracts must enforce on-chain, together with the set of properties that are legitimately delegated to the settlement layer (Section 3).
- **Automated recovery of intent structure from unannotated source.** IntentFuzz identifies a protocol’s intent struct and classifies the semantic role of each field, and separately classifies which functions deposit and which fulfill intents, using only static analysis of Solidity source with no manual annotation. It also flags fill functions gated on a single stored authority as a centralization risk, without requiring code execution. On a benchmark of 9 protocols (5 real-world, 4 synthetic), this recovers the correct struct in 100% of cases (9/9) and classifies deposit and fill functions with 100% recall and 82% combined precision. At corpus scale, across 77 manually labeled contracts, bridge classification reaches 79.5% precision and 97.2% recall, struct selection reaches 88.6% precision and recall, and deposit and fill function classification each reach 100% recall.
- **Automated multi-step fuzz plan synthesis.** IntentFuzz

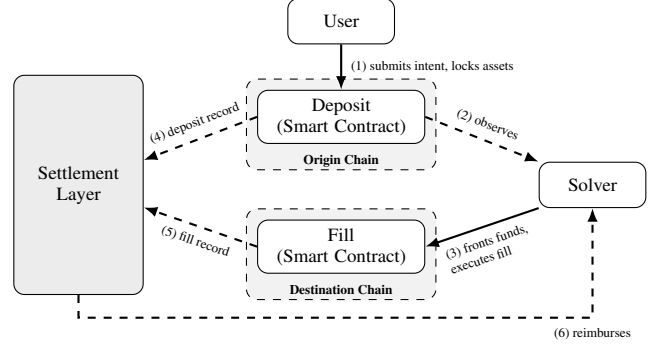


Figure 1: The flow of information in the intent-bridge model

synthesizes multi-step fuzzing sequences directly from the intent structure and function roles recovered by the extraction step described above. This includes a cross-step binding mechanism that lets later steps reference values produced by earlier ones, and a dual-layer tamper mechanism that independently perturbs an ERC-7683 envelope struct and its encoded inner payload to test whether both layers are validated.

- **A three-tier input-generation hierarchy with an LLM-powered feedback loop.** IntentFuzz generates call arguments through a session-intent anchor, followed by type-aware boundary sampling, followed by a large language model (LLM)-assisted recovery step. This step is invoked only when a deposit or fill call fails unexpectedly, and its result is cached per function. In an ablation against real-world deployed contracts, this recovery step raises step-0 execution success from 167/307 (54.4%) to 261/307 (85.0%) and surfaces 5 additional genuine invariant violations. It acts as a cold-start fallback for protocol-specific input requirements that the lower tiers do not anticipate.

Section 2 covers background; Sections 3–5 present the taxonomy, IntentFuzz’s design, and its evaluation; Sections 6–8 discuss limitations, related work, and conclude.

2 Background

In this section, we present the model behind intent-based bridges and the ERC-7683 standard.

2.1 The Intent-Bridge Model

The flow of an intent-based bridge is illustrated in Figure 1.

A user creates an intent (1) by submitting a *deposit* to a smart contract on the chain where the request originates, the *origin chain*. The deposit records the intent’s parameters and locks the assets that back it. (2) A set of entities called *solvers* observe the deposit and compete to (3) fulfill the intent by executing the necessary actions on the *destination chain*. The

fulfillment is also called just *fill* and requires the solver to front its own funds to execute the requested cross-chain transfer. Because the solver fronts its own funds, it must later be reimbursed (6) from the deposit locked on the origin chain. Reconciling the two chains and authorizing this reimbursement is the role of a *settlement layer*, a protocol-specific off-chain-coordinated mechanism that compares the fill against the deposit (4,5) it claims to satisfy. This verification is necessarily retrospective: the settlement layer observes a fill only after it has already been submitted on the destination chain, and it has no opportunity to review or block a fill before execution. Its function is to adjudicate completed cross-chain activity, not to gate it in advance.

2.2 The ERC-7683 Standard

Figure 2 illustrates the ERC-7683 standard [16], which formalizes the interface of intent-based bridges. ERC-7683 standardizes the origin- and destination-chain interfaces of the intent-based bridge model described above. This lets solvers and settlement infrastructure built for one protocol interoperate with any other protocol that implements the standard, rather than requiring bespoke integration per deployment. The standard defines two entry points for deposits on the origin chain, `openFor` and `open`, which accept a `GaslessCrossChainOrder` and an `OnchainCrossChainOrder`, respectively. Both envelopes carry a fill deadline (`fillDeadline`), a type identifier for the inner payload, and the inner payload itself, but they differ in that `GaslessCrossChainOrder` carries additional fields, including a nonce, an origin-chain identifier, and a separate open deadline (`openDeadline`) bounding when the order may be opened at all. The destination-chain fill function receives the same encoding back as `originData`, linking the source- and destination-chain representations of one intent.

The fields that ERC-7683 does not standardize are not part of the envelope itself (e.g., the assets being exchanged, amounts, destination-chain recipients, and any other parameters specific to a given bridge implementation). Instead, the envelope carries them indirectly inside `orderData`, an opaque, Application Binary Interface (ABI)-encoded bytes field shared by both envelope variants. ERC-7683 specifies only that `orderData` exists and that its contents are ABI-encoded; it leaves the internal layout of this inner payload entirely to each protocol’s own implementation. Consequently, the envelope’s directly readable fields are the only ones a generic, protocol-agnostic solver/observer can interpret without further knowledge, decoding `orderData` requires knowing which protocol issued the order. The fact that IntentFuzz can recover the structure of this inner payload from unannotated source code is a key enabler for its protocol-agnostic fuzzing capabilities.

```
function open(OnchainCrossChainOrder order) external;
function openFor(GaslessCrossChainOrder order, bytes signature,
bytes originFillerData) external;
```

```
struct
GaslessCrossChainOrder
{
    address originSettler;
    address user;
    uint256 nonce;
    uint256 originChainId;
    uint32 openDeadline;
    uint32 fillDeadline;
    bytes32 orderDataType;
    bytes orderData;
}
```

```
struct
OnchainCrossChainOrder
{
    uint32 fillDeadline;
    bytes32 orderDataType;
    bytes orderData;
}
```

`orderData` is an opaque bytes field with protocol-specific layout

Origin Chain Contract Interface

Destination Chain Contract Interface

```
function fill(orderId, originData, fillerData) external;
```

Figure 2: ERC-7683’s function signatures and envelope structs.

3 Invariant Taxonomy

Because the settlement layer’s verification is necessarily retrospective (Section 2), this timing constraint splits the properties an intent-based bridge deployment must satisfy into two classes, separated by a single test: is the property *locally decidable* (i.e., checkable using only state already resident on the chain that executes the deposit or fill call, without reference to another chain’s state)? Properties that pass this test can be enforced unilaterally by the origin- or destination-chain contract, at the moment the call executes. If a contract fails to enforce such a property, it is broken, regardless of how the solver behaves or what the settlement layer later concludes. We call a failure to enforce such a property an *invariant violation*. Properties that fail this test, whether the destination-chain recipient, output token, and output amount actually match what the origin-chain deposit specified, are not locally decidable at all: at the moment of the call, the contract has no access to the other chain’s state, so no on-chain check at either endpoint can resolve them. By design, these properties are instead checked by the settlement layer post-fill. A contract’s failure to enforce one of these properties locally is therefore not a defect; it reflects the intended division of responsibility between on-chain enforcement and off-chain settlement. We call this second class a *settlement exposure*.

Table 1 lists the six invariant violations that this taxonomy identifies as properties an intent-based bridge contract must enforce locally. Section 5 reports how well IntentFuzz detects them in practice. Three further properties are intentionally excluded from this taxonomy: whether the destination-chain recipient, output token, and output amount delivered in a fill actually match what the origin-chain deposit specified. These are settlement exposures, not invariant violations, for the same reason given above: verifying any of the cases above would require access to the origin-chain deposit state at the moment

ID	Category	Property Enforced	Consequence of Violation
AC	Access Control	Fill execution is not gated on a single stored authority (e.g., an owner or admin address).	A compromised or malicious authority can unilaterally block or control fills, undermining the open, permissionless solver role.
FRP	Fill Replay Prevention	The same intent cannot be filled twice on the destination chain.	A solver (or attacker) can trigger duplicate payouts against a single locked deposit, draining destination-chain liquidity.
DRP	Deposit Replay Prevention	The same intent cannot be deposited or opened twice on the origin chain.	A single locked deposit can back multiple recorded intents, letting an attacker claim solver reimbursement more than once for a single deposit.
OCB	Origin Chain Binding	The intent’s declared origin chain matches the chain the deposit executes on.	An intent recorded as originating from the wrong chain misleads solvers and the settlement layer about which chain’s deposit backs the fill.
DCB	Destination Chain Binding	The intent’s declared destination chain matches the chain the fill executes on.	A fill can be submitted on a chain the intent never targeted, breaking the correspondence the settlement layer relies on to match fills to deposits.
TB	Temporal Binding	A fill cannot execute after the intent’s deadline has passed.	An intent can be filled after its deadline, forcing acceptance of terms the depositor no longer agreed to, or filling an intent that should instead be refundable.

Table 1: The six invariant violations targeted by IntentFuzz.

of the fill, which is impossible for a destination-chain contract to do.

4 System Design

IntentFuzz turns the invariant taxonomy of Section 3 into an automated detection pipeline that requires no manual annotation of contracts, and no hand-written test harness of the protocol under analysis. IntentFuzz is divided into three phases: Phase 1 statically recovers a protocol’s intent structure and identifies the key functions (deposit and fill); Phase 2 performs a static check for centralized signer dependency (AC); and Phase 3 takes Phase 1 output, builds multi-step fuzz sequences, executes them, and checks their outcomes against the defined invariants (FRP, DRP, OCB, DCB, TB). Phase 3 includes a feedback loop that invokes a large language model to recover protocol-specific input requirements when a deposit or fill call fails unexpectedly. Figure 3 lays out this pipeline end to end.

4.1 Phase 1: Recovering Intent Structure

Struct and Field Discovery. Instead of relying on a human to annotate which struct in a contract represents an intent, and which of its fields carry security-relevant semantics (e.g., an amount, a deadline, a replay-protection identifier), IntentFuzz automatically recovers this structure directly from the contract’s source code. It uses Slither [14], a static-analysis framework for Solidity with Abstract Syntax Tree (AST) and Intermediate Representation (IR)-level extraction, to extract every struct declared in the contract. Each field is then scored against a set of rules that match name hints and Solidity type hints against a small taxonomy of `FieldTypes`, such as `UNIQUENESS`, `TEMPORAL`, `ORIGIN_CHAIN`, and `DEST_CHAIN`.

Each struct’s score is the sum of its fields’ scores, and the top-scoring candidate struct becomes the contract’s canonical intent schema. Hand-crafted scoring is a deliberate choice here, instead of a learning-based model: no labeled corpus of intent structs spanning multiple protocols exists to train one. A rule-based score, moreover, can be inspected field by field to see exactly why a struct was, or was not, selected.

Function Role Classification. The second part of Phase 1 is to identify which functions in the codebase are responsible for creating new intents (deposits) and which are responsible for consuming them (fills). This is a nontrivial problem: a protocol may have multiple deposit or fill functions, and each one with different arguments. The key identifier is the functionality of the function, not its name. Therefore, IntentFuzz classifies each candidate function by analyzing its Slither IR for a combination of signals. These include whether the function writes a new entry into contract storage (e.g., indicating that it records a fresh intent) and whether it emits events (e.g., a deposit function must emit an event that solvers can observe to know a new intent exists). The primary signal, however, is the direction of the token transfers the function performs: whether it receives or sends funds. A function that transfers funds in, typically alongside creating a new state entry, is classified as a deposit; a function that transfers funds out to an identity-typed recipient is classified as a fill; functions matching neither pattern are left unclassified. All possible pairs of deposit and fill functions are then enumerated, and each pair is passed to Phase 3 for fuzzing. An important note here is the looseness of the heuristics: they are designed to be permissive rather than precise (i.e., prioritizing recall over precision), so that no potential deposit/fill pair is missed. Phase 3’s fuzzing and violation detection will later confirm whether a given pair actually behaves as a deposit/fill in practice.

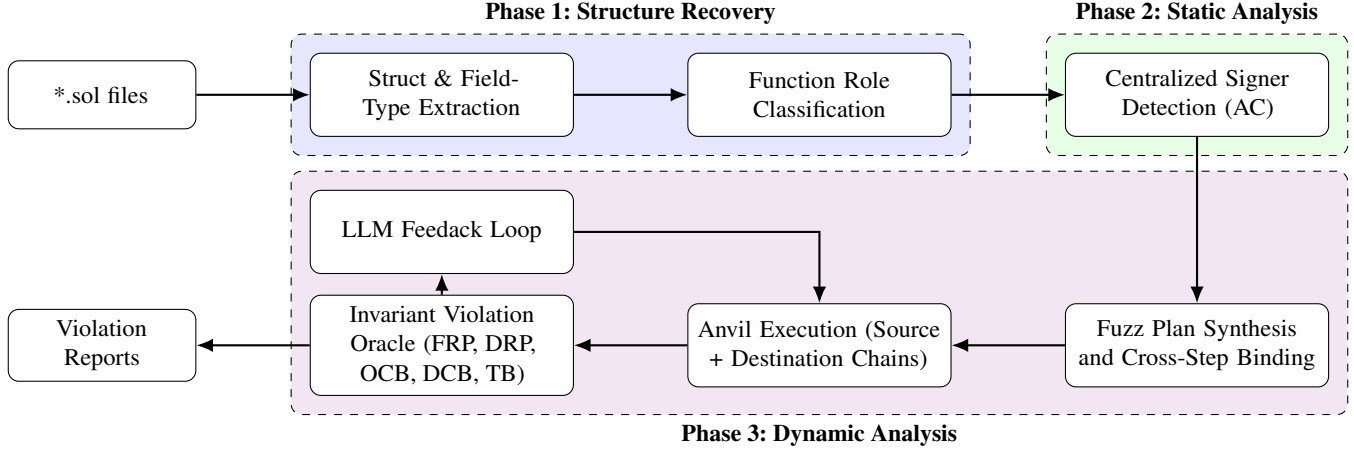


Figure 3: IntentFuzz’s three-phase invariant violation detection pipeline.

4.2 Phase 2: Detecting Centralized Signer Dependency (AC)

Unlike the other five invariants, AC is a static property of the fill function’s code rather than a dynamic property of its execution. AC is triggered when the fill function is gated by a single party. Centralization is the leading root cause behind cross-chain bridge exploits historically [2], and attacks based on compromised private keys still happen in 2026 [18].

IntentFuzz (Figure 3) therefore checks AC without executing the contract, using three static detectors. The first two operate directly on Slither’s AST and IR. One flags a fill function whose access-control check reduces to an equality comparison against a stored, address-typed state variable, the pattern underlying a check such as comparing the caller against a stored owner or admin address. The other flags a fill function that accepts a signature-shaped byte-string parameter and validates it against a single stored authority reached through the function’s internal call chain. The third instead matches on modifier name and argument, since some access-control libraries (e.g., Solady [32]) implement a role check as raw storage-slot arithmetic that produces no IR either detector can recognize as a stored-authority comparison. This third detector flags a fill function gated by a modifier restricting execution to an admin/owner address or to a fixed, centrally-managed role set, such as a permitted solver role.

4.3 Phase 3: Fuzzing and Violation Detection

4.3.1 Fuzz Plan Synthesis

A hand-written fuzz harness would need a test per invariant per protocol, defeating the goal of a tool that works across arbitrary, previously unseen intent-bridge implementations. IntentFuzz (Figure 3) instead synthesizes a *fuzz plan*: an aggregation of every test sequence generated for a contract, based on a set of templates that target each specific invariant

according to the properties present in the intent struct. The number of templates run, therefore, depends on the number of properties/fields identified in Phase 1. For each qualifying deposit/fill pairing identified in Phase 1, the presence of a `UNIQUENESS`-typed field triggers generation of replay-attempt templates targeting FRP and DRP; the presence of `ORIGIN_CHAIN`- or `DEST_CHAIN`-typed fields triggers chain-tamper templates targeting OCB and DCB; and the presence of a `TEMPORAL`-typed field triggers deadline-boundary templates targeting TB, creating and executing intents after the deadline has elapsed. Steps are numbered from 0; step 0 is always the template’s deposit call. For invariants checked at deposit time alone (OCB and TB’s deposit-side variant), this is the step the tamper is applied to directly. For invariants checked later in the sequence (FRP, DRP, DCB, TB’s fill-side variant), step 0 is instead an untampered deposit that a later step binds against or replays.

4.3.2 Cross-Step Parameter Binding

Binding Relation. The easiest way to generate a multi-step call sequence is to generate the parameters freshly in each step independently, but that fails to capture the semantics of a real protocol: a deposit call and the fill call that consumes it must agree on the intent they describe, and a replay-attempt template must resupply the same intent identifier a second time rather than generate a new one. IntentFuzz therefore introduces a *binding* mechanism that lets a later step in a template reference a value from an earlier one, either an argument the earlier step’s call was invoked with or a value the earlier step’s execution generated (e.g., a field of an emitted event). IntentFuzz expresses this as a relation between two steps of the same template:

$$\text{binding}(\text{step}_i, \text{param}) \rightsquigarrow \text{src}(\text{step}_j, \text{arg})$$

$$\text{s.t. } j < i, \quad \text{src} \in \{\text{input}, \text{event}\}$$

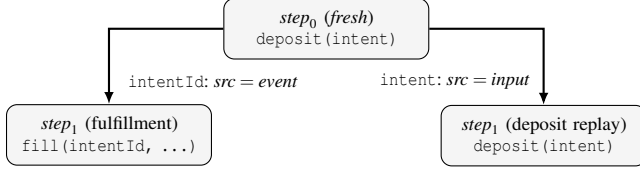


Figure 4: Two templates binding against the same $step_0$ deposit. The fulfillment template’s `intentId` is only ever exposed through the deposit’s emitted event ($src = event$); the replay template’s `intent` argument must be resupplied byte for byte, which no event carries in full, so it instead binds to $step_0$ ’s own recorded input ($src = input$).

where *param* names the call argument $step_i$ must supply, *arg* names either an argument $step_j$ was invoked with ($src = input$) or a field its execution emitted ($src = event$), and $j < i$ restricts a step to referencing only an earlier one. Here, $\rightsquigarrow$ denotes resolution rather than equality: *param* takes on whatever value *arg* held under *src* once $step_j$ has executed. A parameter with no such binding is instead generated independently by the input-generation tiers of the next subsection and marked *fresh*; this is always true of step 0, and of any later parameter whose correspondence to an earlier step cannot be established, a case detailed below.

Choosing a Source. IntentFuzz assigns *src* once, when the template is built, based on what the earlier function’s own event exposes. Deposit and fill functions communicate through events (Figure 1), since an off-chain solver observing a deposit has no access to its calldata, only to what it emitted, so $src = event$ is used whenever the later step’s argument corresponds to a logged event field. $src = input$ is the necessary fallback when an event does not carry a value in full: it exposes only the individual fields a contract chooses to log, an identifier, an amount, never a complete struct argument byte for byte, so a later step needing one can only bind to the earlier step’s recorded input. Figure 4 illustrates both sources on a generic deposit/fill pair; unlike the session-intent anchor described below, whose values are independently generated but held consistent across steps, a binding reuses one specific earlier step’s value exactly.

Establishing Correspondence. Binding is recorded at template-construction time, as a reference to a step, a source, and an argument, before any contract executes. Correspondence is established by static analysis of the earlier function’s own source: a later parameter binds to whichever input argument or logged event field whose name most closely matches it, since Solidity’s ABI normally preserves parameter and event-field names. Matching prefers an exact normalized-name match, falling back to substring or shared-token overlap, without requiring the two functions to share a naming convention beyond this per-field match. This is not always

possible: an ABI-encoded payload passed to a fill call as an opaque byte string, for instance, has no per-field correspondence to the deposit event it is meant to satisfy. When no correspondence can be established, IntentFuzz falls back to constructing the argument directly from the session-intent anchor and the schema it encodes, producing a syntactically valid, self-consistent payload rather than leaving the argument unresolved.

4.3.3 Dual-Layer Tamper Detection

Leveraging the knowledge on ERC-7683’s data structure from Section 2, IntentFuzz can also detect a subtle class of violations. Every template described so far treats a `FieldType` as a single field that is checked at most once, an assumption three ERC-7683 fields violate. `GaslessCrossChainOrder` contains a nonce, an origin-chain identifier, and an open deadline, and `OnchainCrossChainOrder` contains a fill deadline. However, these fields can also be repeated in the protocol-specific `orderData`, which is encoded as a byte string and passed to the deposit function. The contract may check the envelope copy, the inner payload copy, both, or neither. Neither the deadline-boundary template nor the deposit-side deadline template introduced above can distinguish these two copies from a single tampered call: tampering the field once leaves open which of the two, if either, the contract actually validated. For these three fields specifically, IntentFuzz instead generates two variant templates. One tampers only the envelope-level copy, leaving the encoded inner payload untouched; the other decodes the inner payload leveraging `orderDataType`, tampers only its copy, and re-encodes it, leaving the envelope untouched. Comparing the two outcomes tells the oracle, defined at the end of this subsection, whether the protocol validates the envelope copy, the payload copy, both, or neither, a distinction a single-layer tamper cannot draw. This treatment is specific to ERC-7683’s envelope/payload split and to these three fields. A generic deadline field in a protocol with no such split is instead fully covered by the fill-side and deposit-side templates already described.

4.3.4 Input Generation

Fields identified by Phase 1 still need concrete values before a template can execute, and neither purely random values nor a single fixed default suffice. A random byte string offered as a struct-typed deposit argument, for instance, never parses as a valid intent, and values chosen independently for a deposit call and a later fill call within the same template can silently describe two different intents rather than one. IntentFuzz resolves concrete arguments through a three-tier hierarchy:

1. *Session-intent anchor*: a single set of concrete values, generated once per template, from which every step referencing the same schema field draws. This ensures, for example, that a deposit call and the fill call that follows it in

the same sequence agree on the amount, the token, and the deadline they describe, rather than each independently sampling its own.

2. *Boundary-value sampling*: applied whenever no session-intent value applies to a given parameter: values such as zero, one, a maximum safe value, or a chain identifier chosen to match or mismatch the executing fork, selected according to the parameter’s `FieldType`.
3. *LLM-assisted recovery*: invoked only when a call built from the first two tiers unexpectedly reverts, consistent with prior evidence that LLMs can meaningfully assist with smart-contract security tasks more broadly [8]. This excludes calls a template deliberately tampers with, and excludes the intentionally-reverting half of a replay or deadline test, since a revert there is the expected, correct outcome. Once triggered, this tier operates as a feedback loop at two scales: within a single recovery attempt, and across the fuzz plan as a whole. Within an attempt, the Solidity revert reason is passed to an LLM (GPT-4o mini, accessed via GitHub Models) behind a common prompt/response interface. The LLM proposes corrected arguments over a bounded number of retries, each attempt informed by the candidate arguments and revert reasons accumulated over earlier retries in the same attempt. A second, coarser-grained loop sits above the first: the result of the first recovery attempt on a given contract function is cached under that function’s name, so later templates calling the same function reuse it instead of invoking the LLM again (i.e., a single successful repair benefit every template built on that function).

The third tier is best understood as a cold-start fallback for protocols whose input conventions, a nonstandard fee, an unusual encoding, fall outside what the first two tiers were built to anticipate. The LLM’s role is confined to repairing one specific failed call, not to rank templates or decide whether a violation occurred. Section 5 reports the contributions of this tier.

4.3.5 Execution and Oracles

Each template ultimately becomes a sequence of transactions submitted to Anvil, a local Ethereum node, running either a fresh deployment compiled directly from the target contract’s source or an existing on-chain deployment reached directly through its ABI and exercised against forked chain state. A template-type-specific oracle then determines whether the sequence’s outcome constitutes a violation. A replay-attempt template checks whether a second call carrying the same intent identifier succeeded where FRP or DRP requires it to have reverted. A chain-tamper template checks whether a call carrying a deliberately mismatched chain identifier succeeded where OCB or DCB requires a revert. Either deadline template, fill-side or deposit-side, checks whether the tampered

call succeeded where TB requires a revert. The replay oracle in particular has to guard against a protocol-level confound: some protocols assign each deposit an auto-incrementing identifier rather than accepting one the caller supplies. A naive check would therefore flag two structurally identical deposit calls as a replay, even though they produced two distinct, non-colliding intents. IntentFuzz’s oracle instead inspects the identifier each call actually produced and reports a violation only when both calls in the same replay template resolve to the *same* identifier. This is the one condition under which a repeated call is a genuine replay rather than an artifact of how the protocol assigns identifiers.

The chain-tamper and deadline oracles face a different confound: a tampered value is only evidence of a missing check if the call actually carries that value into the executed transaction. A target field may have no counterpart on the function under test at all, present only on a struct or an encoded payload used elsewhere in the protocol. In that case, the tamper is well-formed but has nowhere to attach, and the resulting call is indistinguishable from an untampered one. Reporting a violation from such a call would be a false positive: the check was never exercised, let alone found missing. IntentFuzz therefore verifies, for every tampered step, that some parameter of the invoked call actually carries the targeted field, directly, as part of a struct argument, or as part of an encoded payload, before allowing its oracle to report a violation. A tamper that cannot attach anywhere is instead recorded as inconclusive, preventing a well-formed but inert tamper on a deployed contract from being misreported as a passed check when the targeted field simply has no counterpart in that contract’s call.

5 Evaluation

This section evaluates IntentFuzz’s ability to recover a protocol’s intent structure (Phase 1) and detect the invariant violations of Table 1 (Phases 2–3). We first evaluate IntentFuzz against synthetic ground truth with known struct, function roles, and planted bugs. We evaluate it against deployed contracts, where ground truth comes from manual review.

5.1 Corpus Construction

In this work, we assembled a corpus of intent-bridge Solidity contracts from GitHub and filtered down in stages as shown in Figure 5. A crawler first collects candidates via GitHub’s code-search API using seventeen intent-bridge-specific Solidity keyword patterns. Because code search matches any file containing these terms anywhere in a repository, a path-based filter removes test, mock, interface, library, and build-artifact files that surface only through incidental keyword matches. A stricter keyword threshold, requiring at least one file to match more than two patterns, then separates repositories with genuine intent-bridge content from those with only incidental overlap, such as token variants, AMMs, and gover-

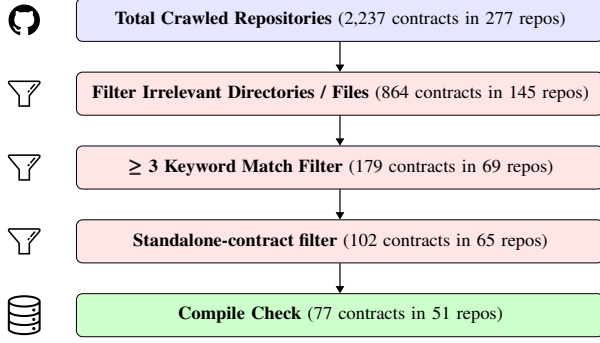


Figure 5: Corpus construction pipeline and filtering stages for the evaluation corpus.

nance tooling that share vocabulary without implementing the pattern. Before Phase 1 analysis, a standalone-contract filter removes files that are abstract contracts, thin wrappers, or chain-specific inheritance extensions of another in-repo contract. Of the 102 contracts left, 25 failed to compile for reasons external to IntentFuzz (e.g., missing upstream package dependencies, broken repository build configurations), and are excluded from evaluation. The remaining 77 contracts were manually assigned ground-truth labels (a bridge/not-bridge verdict, the correct intent struct where applicable, and the correct deposit and fill function sets) against which the following subsection reports Phase 1’s corpus-scale accuracy.

5.2 Coverage Measurement

Generic code coverage does not fit invariant testing. Instruction coverage marks a `JUMPI` covered on mere execution regardless of outcome, while branch coverage requires both outcomes to be observed; but for a security check, the failing branch *is* the violation, so requiring both outcomes ties full branch coverage to the presence of a bug. IntentFuzz instead reports *guard-JUMPI coverage*: the fraction of `JUMPI` sites guarding a `REVERT` that were visited, regardless of outcome. Guards are identified directly on deployed bytecode. A breadth-first traversal of the control-flow graph, with an abstract-stack interpreter tracking `PUSH/DUP/SWAP/POP` to resolve jump targets beyond literal pushes, flags any `JUMPI` whose taken branch or direct fall-through reaches `REVERT`; this is the pattern `solc` emits for non-payable and argument-validation checks. Visitation, from an EVM step trace, is computed per entry point (selector resolved via `evmole` [24]) scoped to only its reachable guards, with proxies resolved first. Section 5.7 reports these figures alongside violation counts.

5.3 Phase 1: Struct Identification and Function Classification

First, we describe the evaluation setup and then report Phase 1’s evaluation results.

5.3.1 Evaluation Setup

Phase 1’s struct-selection and function-classification accuracy is measured at two scales: (1) a nine-protocol benchmark produced by the authors to serve as a labelled baseline with fully known ground truth, and (2) the corpus described in Section 5.1, where ground truth was instead established through manual review. The benchmark comprises five real-world intent bridges (deBridge DLN [9], Across SpokePool [1], Across ERC-7683, Omni SolverNet [28], Relay Depository [29]) and four synthetic mock baselines implementing the deposit/fill pattern directly (native-ETH and ERC-20 variants, with and without ERC-7683 envelope encoding), giving every classification decision an unambiguous ground truth.

5.3.2 Results

Table 2 reports the results on the nine-protocol benchmark: Phase 1 selects the correct intent struct in all nine protocols and achieves 100% recall on both deposit- and fill-function classification, at 82% combined precision (33 true positives, 7 false positives, 0 false negatives).

Task	TP	FP	FN	Precision	Recall	F1
Struct	9	—	—	100%	100%	100%
Deposit	18	5	0	78%	100%	88%
Fill	15	2	0	88%	100%	94%
Combined	33	7	0	82%	100%	90%

Table 2: Phase 1 accuracy on the nine-protocol benchmark. Struct selects exactly one candidate per protocol, so no false-positive/false-negative decomposition applies.

Table 3 reports the same accuracy measures over the 77 manually labeled corpus contracts described in Section 5.1. If Phase 1 misses to classify a contract as a bridge (a bridge false negative) it yields no struct or function classification for Phase 1 to be scored on. On the other hand, a contract misclassified as a bridge in Phase 1 (a bridge false positive), has no ground-truth struct or function set to score its output against. For this reason, struct-selection, deposit, and fill accuracy are computed only over the true-positive bridges, the 35 contracts on which Phase 1’s verdict and the ground-truth verdict agree that the contract is a bridge. Of these 35 contracts, all carry a struct ground-truth annotation added by the authors, 31 carry a deposit annotation, and 30 carry a fill annotation. Because deposit and fill accuracy are themselves scored at the function level, and a single contract may contribute more than one ground-truth deposit or fill function, the per-task sample sizes are not identical. At corpus scale, Phase 1 again achieves 100% recall on both deposit- and fill-function classification.

The two false-positive populations differ in composition. At corpus scale, bridge-classification false positives are pre-

Task	TP	FP	FN	TN	Precision	Recall	F1
Bridge classification	35	9	1	32	79.5%	97.2%	87.5%
Struct selection	31	4	4	–	88.6%	88.6%	88.6%
Deposit	55	21	0	–	72.4%	100%	84.0%
Fill	31	14	0	–	68.9%	100%	81.6%

Table 3: Phase 1 accuracy over the 77 manually labeled corpus contracts. TN is defined only for bridge classification, the only task with a genuine negative class.

dominantly adapter or forwarder contracts that call out to an external, separately deployed bridge contract without owning any on-chain intent state themselves. This shape is indistinguishable from a genuine deposit under the criteria Phase 1 uses to identify one, even though the contract is not a bridge in its own right. Deposit- and fill-function false positives within correctly classified bridges are predominantly cross-chain message-handler callbacks: functions a cross-chain messaging protocol invokes to deliver an incoming message. These write state, move funds, and emit events much like a genuine deposit or fill, but they are related to the settlement-layer, not user- or solver-facing entry point. A smaller residual (e.g., auction/bid functions) does not fit either pattern.

Please note that IntentFuzz prioritizes recall over precision on deposit- and fill-function classification by design. If Phase 1 fails to classify a function, it never enters a fuzz plan, so no invariant test run for that function. A falsely classified function, in contrast, only costs Phase 3 a wasted fuzzing sequence, which its execution oracle filters out.

Main Insights. IntentFuzz recovers the correct intent struct and every genuine deposit/fill function with 100% recall at both benchmark and corpus scale. Its remaining false positives cluster in two explainable patterns, adapter/forwarder contracts and cross-chain message-handler callbacks.

5.4 Phase 2: Centralized Signer Dependency Detection

In Phase 2, IntentFuzz detects whether a fill function is gated on a single stored authority, which poses a centralization risk. This section first describes the evaluation setup and then reports the results.

5.4.1 Evaluation Setup

IntentFuzz first validates AC detection against a small set of synthetic contracts constructed for this purpose: three variants gating fill execution behind a centralized authority (a stored owner address, a stored verifier address, and an ERC-7683 variant of the same pattern) and four secure baselines with no such gate. Phase 2 is evaluated on a broader real-world corpus: since it requires no forked chain state, it was run over 35 protocol/chain registrations drawn from this work’s real-world corpus. This resolves to 21 unique underlying contract

sources once the same source analyzed across multiple chain deployments is counted once.

5.4.2 Results

IntentFuzz flags all three vulnerable variants and none of the four baselines (3/3 recall, 4/4 true negatives). On the crawled corpus, IntentFuzz flags 4 of the 21 contracts with an AC finding, each manually verified against the deployed source before being reported here. The remaining 17 are clean. Table 4 lists the four flagged contracts together with the gate each detector matched: a stored-authority signature check, a direct stored-address equality comparison, and a permissioned-role modifier.

Protocol	Fill-Side Gate	Detection Mechanism
Relay Depository (Base)	allocator. isValidSignatureNow(...)	Stored-authority signature check (IR)
Phatdtd/simple-bridge SimpleBridge	require(msg.sender == owner())	Stored-address equality check (IR)
Omni SolverNet (Ethereum mainnet)	onlyRoles(SOLVER) modifier	Modifier name/argument inspection
omni-network/omni SolverNetOutbox (chain 421614)	onlyRoles(SOLVER) modifier	Modifier name/argument inspection

Table 4: AC findings on the real-world corpus. Each finding was manually verified against the deployed contract’s source.

Main Insights. IntentFuzz’s three static detectors achieve perfect recall on the synthetic gated/ungated contracts (100%) and, without executing any code, flag 4 of 21 real-world protocols with a manually confirmed centralized-signer dependency.

5.5 Phase 3: Detection Accuracy on the Synthetic Dataset

Phase 3’s invariant-violation recall is measured via mutation testing against the four synthetic mock baselines introduced in Section 5.3 (native-ETH and ERC-20 token transfer, with and without ERC-7683 envelope encoding). For each baseline, this work derives a set of mutants by removing exactly one invariant-enforcing `require()` guard per mutant (for example, a check against a replay-protection mapping prior to a fill, or a chain-identifier equality check), leaving the underlying state write intact. Group sizes are asymmetric: the ERC-7683 native-ETH baseline contributes 8 mutants (the only group exhibiting the dual-layer envelope/payload variants described in Section 4), while the remaining three baselines contribute 5 mutants each, for 23 mutants in total. Each contract is compiled and deployed locally. Table 5 shows the results.

IntentFuzz flags all 23 mutants as invariant violations and reports zero violations when run against the four unmutated baselines, yielding 100% recall and 100% precision.

Invariant	Total Mutants	Mutants Found
FRP (fill replay)	4	4
DRP (deposit replay)	4	4
OCB (origin chain binding)	6	6
DCB (destination chain binding)	4	4
TB (deadline enforcement)	5	5
Total	23	23

Table 5: Planted-mutant recall by invariant.

5.6 Phase 3: Real-World Evaluation

The evaluation setup for Phase 3 evaluation is described first, followed by representative findings.

5.6.1 Evaluation Setup

Real intent-based bridge deployments carry protocol-specific initialization, such as role grants, cross-chain messaging registration, and linked periphery contracts, that a generic local redeployment cannot reconstruct. Rather than compiling and redeploying each of the corpus’s 77 contracts locally, as in Section 5.5’s synthetic mock evaluation, this evaluation forks live chain state and reaches each contract through its real, already-deployed address and ABI. Addresses come from scanning each repository’s own deploy-script artifacts, confirming deployed bytecode on the target chain before use. This yields 24 protocol/chain deployments, each run under two input-generation configurations: a baseline (B) using only IntentFuzz’s heuristic tiers, and an LLM-assisted (L) configuration adding the recovery tier of Section 4.

A live, correctly initialized address alone does not make a contract exercisable: executing a deposit function still requires funding the test account with correct token balances and approvals, and, for some deployments, working around an external dependency, such as a cross-chain messaging system.

5.6.2 Results

Under the baseline configuration, IntentFuzz executed 464 templates across the 24 deployments and confirmed 17 genuine invariant violations, spanning eight vulnerable GitHub repositories, each finding reproducible against the public, deployed bytecode. Below, we present three examples of findings with snippets of code to illustrate the missing checks. After, we show the invariant violation breakdown by category.

TB (Deposit-Side) – Non-Envelope with Undecided Expiration. eco/eco-routes’ [12] Portal contract, on Ethereum mainnet, decodes an incoming order’s route deadline as part of its `OrderData` and passes it straight into intent creation without ever comparing it to the current time:

```
OrderData memory orderData =
```

```
abi.decode(order.orderData, (OrderData));
// FIX: require(orderData.routeDeadline >
    block.timestamp, "expired");
(bytes32 orderId, ) = _publishAndFund(
    orderData.destination, orderData.route,
    orderData.reward, false, msg.sender
);
```

Neither `publish` nor `_fundIntent`, the two calls `_publishAndFund` makes internally, reference `orderData.routeDeadline` at any point; an intent whose route deadline has already elapsed is created and funded exactly like a valid one. No solver can ever fill it, but the deposit is accepted and the funds are locked permanently.

TB (Deposit-Side) – Envelope-Only with Unvalidated ERC-7683 Envelope Deadline. BootNodeDev’s [5] contract resolves an incoming `OnchainCrossChainOrder` envelope’s fill deadline into its inner order record with no accompanying check:

```
OrderData memory orderData = OrderEncoder.decode(
    _orderData);
(...)
// FIX: require(_fillDeadline > block.timestamp,
    "expired fillDeadline");
orderData.fillDeadline = _fillDeadline;
(...)
orderId = OrderEncoder.id(orderData);
(...)
```

IntentFuzz’s dual-layer tamper mechanism (Section 4) isolates this gap to the deposit path: `_resolvedOrder` never checks that `_fillDeadline` lies in the future before writing it into `orderData.fillDeadline` and accepting the deposit. After a manual analysis, we check that the fill path (i.e., the other smart contract on the destination chain) correctly rejects a fill once this deadline has passed. However, the presence of that check in the destination chain cannot prevent an already-expired intent from being funded in the origin chain; it can only guarantee that intent is never filled. A deposit with an already-elapsed `_fillDeadline` therefore locks its funds permanently. This same gap recurs across several deployments.

DRP – Deposit Replay via a Missing Status Check. decentxyz’s GhostGateway [10], on Arbitrum, derives its order identifier deterministically from the deposited order’s contents through a function `_getOrderId` and writes the resulting record without first checking whether that identifier’s status is already set:

```
bytes32 orderId = _getOrderId(orderData);
// FIX: require(orderStatus[orderId] ==
    OrderStatus.UNFILLED, "already opened");
orders[orderId] = orderData;
orderStatus[orderId] = OrderStatus.OPENED;
```

Invariant	B	L	Arch. (B)	Arch. (L)
DRP (deposit replay)	4	9	2	2
TB, deposit-side (non-env.)	4	4	10	10
TB, deposit-side (env.-only)	9	9	—	—
Total	17	22	12	12

Table 6: Confirmed invariant violations by category, baseline (B) vs. LLM-assisted (L), including Across SpokePool’s architectural count under each configuration (Arch.).

```
(...)
IERC20(TypeCasts.bytes32ToAddress(orderData.
    inputToken)).safeTransferFrom(
    msg.sender, address(this), orderData.amountIn
);
```

Two deposit calls carrying identical order contents therefore produce the same `orderId`, and GhostGateway’s `open` function accepts the second call, silently overwriting the first order’s record and re-emitting the opening event.

Violation Breakdown. Table 6 decomposes the 17 baseline violations by category.

- **DRP (4 Instances).** One each on Relay Depository and GhostGateway (the first finding above), and two on eco/eco-routes’ Portal contract, whose `open` and `openFor` entry points each independently permit deposit replay.
- **TB – Deposit-Side with Non-Envelope (4 Instances).** All four recur on eco Portal, through two distinct internal paths each on `open` and `openFor` (the second finding above).
- **TB – Deposit-Side with Envelope-Only (9 Instances).** GhostGateway contributes 1; BootNodeDev’s two ERC-7683-based implementations, together with the banr1 [4]/Kaushikh76 [21] deployments, contribute 2 each (6 in total); and MukulKolpe/ETHGlobalTaipei [26] and SarveshLimaye/trifecta [31] contribute 1 each (2 more).

Across SpokePool’s 12 architectural instances split into TB’s 10 non-envelope instances and DRP’s 2 `unsafeDeposit`-only instances. Manual review of the deployed source confirms each is a documented protocol choice: `_depositV3` validates only an upper bound on `fillDeadline`, and the contract’s `INFINITE_FILL_DEADLINE` sentinel, used when off-chain validators query for expired deposits, confirms an already-elapsed deadline at deposit time is an anticipated case. `unsafeDeposit` similarly accepts a caller-supplied intent identifier by design, flagged as such in its own name and documentation. Guard-JUMPI coverage show that these are exercised code paths: 82.9% of deposit-side and 91.4% of fill-side guards are reached on both Across chains.

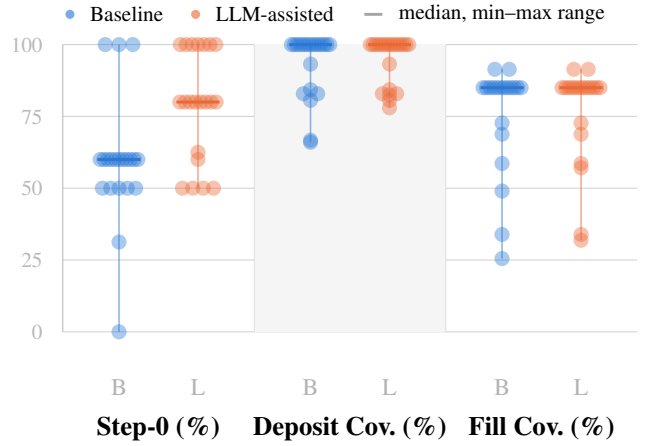


Figure 6: Step-0 reachability and guard-JUMPI coverage (Section 5.2), baseline (B) vs. LLM-assisted (L).

Low Fill-Side Coverage. Low fill-side coverage in this evaluation traces to three distinct causes. Of the seven protocol codebases with low or entirely unmeasured fill-side coverage, four, eco Portal and the three dormant corpus deployments below, are low because no call could be constructed or dispatched at all. Two more, Omni SolverNet and Relay Depository, instead reflect a confirmed access-control gate. The seventh, deBridge DLN, is neither: its fill call is constructed and dispatched but fails a fork-specific value check that neither boundary sampling nor LLM correction can supply (detailed below). Figure 6 plots step-0 reachability and guard-JUMPI coverage under both configurations across the 21 of 24 deployments that completed at least one successful call under either configuration, baseline (B) alongside its LLM-assisted (L) counterpart discussed in Section 5.7. Each dot is one deployment, jittered to reduce overlap, with the tick marking the median.

- **Confirmed Access-Control Gates.** Every fill call against Omni SolverNet is correctly rejected by the same `onlyRoles(SOLVER)` gate Section 5.4 flags under AC (Table 4). Coverage there reflects the union of guard checks reached across several differently-shaped rejected calls hitting this gate. This fork evaluation independently corroborates, dynamically, the same centralized-authority dependence Phase 2 already identified statically. Relay Depository’s fill-side guards are unreachable for the same reason: `execute()` requires a valid EIP-712 signature from its stored allocator authority, the same AC finding Table 4 reports, which this evaluation cannot forge.
- **Eco/Eco-Routes’ Portal.** Only one of three fill-role candidates (the generic ERC-7683 fill) matches a call the tool can construct, while the two protocol-specific functions, `fulfill` and `fulfillAndProve`, genuinely deployed but

built around a different, `Route-struct`-based signature, never dispatch a transaction at all.

- **Dormant Corpus Deployments.** `Omni SolverNetOutbox` and `OpenGateV2` [11] never reach a fill call at all (1 and 2 templates, respectively): their deposit step reverts. These hackathon-style deployments depend on on-chain protocol state, cross-chain messaging registration among them, that a generic fork cannot reconstruct.
- **deBridge DLN.** `fulfillOrder` requires the order's `takeChainId` field to match the executing fork's actual chain identifier, a value boundary-value sampling does not reliably supply; a large share of fill calls revert before reaching any invariant check. LLM-assisted recovery does not close this either, since a fork-specific chain identifier is not something a Solidity revert message can supply.

5.6.3 Performance Metrics

Static extraction (Phase 1) takes a median 0.47s per contract across 27 mock contracts (23 mutants, 4 baselines; end-to-end evaluation there (Phases 1-3) takes a median 0.88s per contract, over which 273 templates (507 transactions) execute in a median 14ms, with no network round-trip. Phase 3 alone, on real-world crawled contracts over public RPC, takes a median 26.4s per deployment (varies with template count): 1.7s/template and 0.9s/transaction, roughly two orders of magnitude slower, since each transaction costs a very slow public RPC round-trip plus a `debug_traceTransaction` call for guard-JUMPI coverage. The smallest genuine finding, Relay Depository's `DRP`, is confirmed in 3.8s (0.96s/template, 6 transactions) against an already-forked chain; a larger deployment, `BootNodeDev's Hyperlane7683`, runs 17 templates in 31.16s (1.83s/template, 26 transactions).

Main Insights. Forking 24 live protocol deployments, `IntentFuzz` confirms 17 genuine invariant violations across 8 vulnerable GitHub repositories, each independently reproducible against public deployed bytecode, while correctly separating `Across SpokePool's` 12 flagged instances as a documented architectural choice.

5.7 Contribution of LLM Feedback Loop

Overall Impact. The LLM-assisted recovery tier enabled on the same 24 deployments raises step-0 success (the fraction of deposit/fill calls that execute successfully rather than reverting before invariants are tested), from 167/307 (54.4%) to 261/307 (85.0%), over 81 LLM recovery calls (with a total cost $\approx \$0.05^1$), and raises the invariant-violation count from 17 to 22 (cf. Table 6).

All 5 new violations are `DRP` (deposit replay), reached through native-currency deposit paths the baseline configuration could not: `eco/eco-routes' Portal` contract's `fund`,

¹pricing of \$0.15 per million input tokens

`fundFor`, `publishAndFund`, and `publishAndFundFor` entry points (4), and Relay Depository's `depositErc20` entry point (1). The latter mirrors the already-confirmed replay on its `depositNative` counterpart. Reaching each entry point requires supplying the correct native fee or token-transfer amount the call expects, a value neither the session-intent anchor nor boundary-value sampling can infer from the ABI alone. This tier also lets `fillRelay` and `fillV3Relay` succeed on `Across SpokePool` by correcting its multi-field `relayData` struct, raising step-0 success on this deployment to 35/35; its architectural total is unchanged at 12.

Coverage Gains and Their Limits. `deBridge DLN` illustrates this tier's contribution: step-0 success rises from 0% to 63%, deposit-side coverage rises from 66.0% to 78.0% and fill-side coverage from 49.0% to 57.1%. `eco/eco-routes' Portal` shows the same pattern at larger scale: step-0 success rises from 31% to a fully saturated 100%, and deposit-side coverage from 66.7% to 82.5%.

Deposit- vs. Fill-Side Asymmetry. This tier's contribution concentrates on protocols whose deposit and fill functions expect input conventions, a native fee, a fork-specific gas or messaging cost, that the first two input-generation tiers cannot get from the intent struct's field types. This substantiates the cold-start framing of Section 4.3.4. Its effect on fill-side reachability is narrower: aside from `Across` and `eco/eco-routes` just discussed, fill-side guard-JUMPI coverage is unchanged across every other protocol. Section 6 discusses why this asymmetry is expected: a Solidity revert reason is a causal signal that often names the value the failing check expected, which is why this tier succeeds on self-contained deposit-side calls. A fill call in most protocols examined, by contrast, is missing structured values, an `orderId` or a re-encoded payload, that exist only in the deposit transaction's event log.

Main Insights. The LLM-assisted tier deepens deposit-side testing, raising step-0 success from 54.4% to 85.0% and surfacing five additional genuine `DRP` violations by inferring protocol-specific fees and amounts directly from transaction revert reasons.

6 Discussion

Coverage Only as a Measurement Tool. A natural question is why the guard-JUMPI coverage measurement already collected (Section 5.2) is not fed back to drive input generation directly. The evaluation already tests a strictly more capable version of this idea: the LLM-assisted recovery tier also proposes new arguments for a failed call, guided by the actual revert reason rather than only a covered/uncovered signal, yet Section 5.7 reports its effect on fill-side coverage is unchanged across nearly every protocol family. This is because the residual fill-side gaps are not argument-guessing problems: most fill arguments are already resolved directly

through the cross-step binding mechanism of Section 4, and the rest fail for structural reasons, an access-control gate, on-chain state a generic fork cannot reconstruct, or a fill signature with no counterpart in the recovered schema, none of which a better-guided guess, coverage-driven or otherwise, can supply. A coverage-guided search loop would therefore add search overhead without adding tested surface area.

Fill Functions with Protocol-Specific Argument Shapes.

IntentFuzz’s fill-argument construction targets the parameters a generic ERC-7683 fill call carries, `fill(orderId, originData, fillerData)`, or their equivalent recovered from the deposit-side intent schema; it cannot construct arguments for a fill entry point whose parameters have no such counterpart. `eco/eco-routes`’ Portal contract illustrates this: alongside the generic fill entry point, it also exposes protocol-specific `fulfill/fulfillAndProve` functions taking a `Route` struct, a prover address, and a `sourceChainDomainID`, none of which corresponds to any field in the recovered intent schema. Closing this gap would mean hardcoding `eco-routes`’ own proof-hash formula `(keccak256(abi.encodePacked(chainId, keccak256(abi.encode(route)), rewardHash)))` directly into the harness: unlike ERC-7683’s `orderData` encoding, which is standardized and derivable from the recovered schema alone, this construction is bespoke to `eco-routes`, and hardcoding it is exactly the per-protocol special-casing IntentFuzz is designed to avoid. The LLM-assisted recovery tier cannot close this gap either, since it repairs calls by inferring arguments from revert reasons. This check instead requires reproducing a deterministic hash’s exact byte-level preimage, and no language model can guess a 32-byte digest into agreement with a value it cannot itself compute.

7 Related Work

No existing tools detect the invariant violations of Table 1 without a hand-written, protocol-specific oracle, since no tool models the deposit/fill/settlement structure that intent-based bridges assume. Coverage-driven fuzzers, Echidna [17]/Medusa [34] (property-based) and Foundry fuzz [15] (unit-test), only flag a violation of a property already encoded as an assertion. Foundry fuzz additionally has no mechanism for the multi-step `deposit`→`fill`→`replay` sequences or ERC-7683’s envelope/inner-payload structure these invariants require. Recent work on LLM-guided fuzzing [13, 20, 27] targets general software, JavaScript engines, and embedded firmware rather than smart contracts. Static analyzers such as Slither [14] instead match known-bad code patterns (reentrancy, arbitrary-send-eth, timestamp dependence), i.e., removing a `require()` guard that enforces a protocol-level invariant leaves no new pattern to match, so a pattern-matching detector cannot, by construction, recognize the resulting *absence* of a check. Even

fuzzers built for stronger coverage, such as EFCF [30] (cross-contract interaction coverage) and ConFuzzius [33] (concolic execution), remain confined to a single chain, with no mechanism for cross-chain replay or chain-binding sequences.

A second line of work targets bridge contracts directly rather than generic Solidity, but not intent-based bridges specifically: BridgeGuard [39] uses symbolic dataflow analysis to find missing relayer/oracle authentication; SmartAxe [23] infers access-control patterns and builds cross-chain control-/data-flow graphs to catch semantic inconsistencies; BridgeFuzz [36], motivated by prior tools’ single-contract focus missing the cross-chain interaction chain, targets the hybrid on-chain-contract/off-chain-relayer architecture to detect balance mismatches, protocol errors, and off-chain denial-of-service bugs. All three are built on the original bridge model that intent-based bridges replace. Since none of the solutions above model the deposit/fill/settlement structure central to intent-based bridges, nor the invariants in Table 1, we do not attempt an empirical comparison.

Finally, work on intent-based bridges is still scarce. Beyond some empirical papers on this topic, [3] has examined the economic exploitation of solver liquidity.

8 Conclusion

This paper presented IntentFuzz, a protocol-aware fuzzer for intent-based bridges, and four concrete contributions. First, a taxonomy separating invariant violations from settlement exposures (the latter, are delegated to the off-chain settlement layer and are out of scope). Second, IntentFuzz recovers a bridge’s intent structure and deposit/fill function roles directly from unannotated Solidity source, at 100% struct-recovery and function-classification recall and 82% combined precision on a 9-protocol benchmark (5 real-world, 4 synthetic), and 97.2%/88.6% bridge-classification and struct-selection recall with 100% deposit/fill recall at corpus scale (Section 5.3). Third, it synthesizes multi-step fuzz sequences from the recovered structure alone, including a cross-step binding mechanism that carries values between steps and a dual-layer tamper mechanism that isolates whether an ERC-7683 envelope, its encoded inner payload, or both are actually validated (Section 4). Fourth, a three-tier input-generation hierarchy, anchored on a shared session intent, backed by boundary-value sampling, and falling back to an LLM-assisted recovery step. Across synthetic ground truth and 24 production intent-based bridge deployments, IntentFuzz finds 22 real invariant violations independently reproducible against publicly deployed bytecode, without any hand-written or protocol-specific properties. These results indicate that automatically recovering intent-based bridge structure and using it to drive fuzzing is both practical and effective at surfacing genuine, exploitable weaknesses in the intent-based bridge ecosystem as it is deployed today.

References

- [1] Across Protocol. across-protocol/contracts. <https://github.com/across-protocol/contracts>, 2026. GitHub repository, accessed 2026.
- [2] André Augusto, Rafael Belchior, Miguel Correia, André Vasconcelos, Luyao Zhang, and Thomas Hardjono. SoK: Security and privacy of blockchain interoperability. In *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*, 2024.
- [3] André Augusto, Christof Ferreira Torres, André Vasconcelos, and Miguel Correia. Exploiting liquidity exhaustion attacks in intent-based cross-chain bridges. *arXiv preprint arXiv:2602.17805*, 2026.
- [4] banr1. banr1/forked-intents-framework. <https://github.com/banr1/forked-intents-framework>, 2026. GitHub repository, accessed 2026.
- [5] BootNodeDev. Bootnodedev/intents-framework. <https://github.com/BootNodeDev/intents-framework>, 2026. GitHub repository, accessed 2026.
- [6] Arjun Chand. Li.fi - with intents, it's solvers all the way down. URL: <https://li.fi/knowledge-hub/with-intents-its-solvers-all-the-way-down>.
- [7] Tarun Chitra, Kshitij Kulkarni, Mallesh Pai, and Theo Diamandis. An analysis of intent-based markets. *arXiv preprint arXiv:2403.02525*, 2024.
- [8] I. David, L. Zhou, K. Qin, D. Song, L. Cavallaro, and A. Gervais. Do you still need a manual smart contract audit? (arXiv:2306.12338v2), June 2023.
- [9] deBridge Finance. debridge-finance/dln-contracts. <https://github.com/debridge-finance/dln-contracts>, 2026. GitHub repository, accessed 2026.
- [10] decentxyz. decentxyz/decentents. <https://github.com/decentxyz/decentents>, 2026. GitHub repository, accessed 2026.
- [11] dimasriat. dimasriat/fuelstack. <https://github.com/dimasriat/fuelstack>, 2026. GitHub repository, accessed 2026.
- [12] eco. eco/eco-routes. <https://github.com/eco/eco-routes>, 2026. GitHub repository, accessed 2026.
- [13] J. Eom, S. Jeong, and T. Kwon. Fuzzing javascript interpreters with coverage-guided reinforcement learning for LLM-based mutation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1656–1668, 2024.
- [14] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A static analysis framework for smart contracts. In *Proceedings of the 2nd IEEE/ACM International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15, Montreal, Canada, 2019. doi:10.1109/WETSEB.2019.00008.
- [15] Foundry Contributors. Foundry: A blazing fast, portable, and modular toolkit for Ethereum application development. <https://github.com/foundry-rs/foundry>, 2024. Accessed 2026.
- [16] Francisco Giordano, Mark Toda, Matt Rice, Nick Pai, Alexander Lindgren, Mark Gretzke, and Chris Cashwell. ERC-7683: Cross chain intents. Ethereum Improvement Proposals, no. 7683, April 2024. Draft. URL: <https://eips.ethereum.org/EIPS/eip-7683>.
- [17] Gustavo Grieco, Will Song, Artur Cygan, Josselin Feist, and Alex Groce. Echidna: Effective, usable, and fast fuzzing for smart contracts. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 557–560, Virtual Event, USA, 2020. ACM. doi:10.1145/3395363.3404366.
- [18] Halborn. Explained: The iotex hack (february 2026). <https://www.halborn.com/blog/post/explained-the-iotex-hack-february-2026>, 2026.
- [19] Bo Jiang, Ye Liu, and W. K. Chan. Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, pages 259–269, Montpellier, France, 2018. ACM. doi:10.1145/3238147.3238177.
- [20] Y. Jiang, J. Liang, F. Ma, Y. Chen, C. Zhou, Y. Shen, Z. Wu, J. Fu, M. Wang, S. Li, and Q. Zhang. When fuzzing meets LLMs: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 492–496, 2024.
- [21] Kaushikh76. Kaushikh76/intents-framework. <https://github.com/Kaushikh76/intents-framework>, 2026. GitHub repository, accessed 2026.
- [22] Sung-Shine Lee, Alexandr Murashkin, Martin Derka, and Jan Gorzny. SoK: Not quite water under the bridge: Review of cross-chain bridge hacks. In *Proceedings of the 2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, Dubai, UAE, 2023. doi:10.1109/ICBC56567.2023.10174993.

- [23] Zeqin Liao, Yuhong Nan, Henglong Liang, Sicheng Hao, Juan Zhai, Jiajing Wu, and Zibin Zheng. SmartAxe: Detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis. *Proc. ACM Softw. Eng.*, 1(FSE), July 2024. doi:10.1145/3643738.
- [24] Maxim Andreev. cdump/evmole. <https://github.com/cdump/evmole>, 2026. GitHub repository, accessed 2026.
- [25] Barton P. Miller, Lars Fredriksen, and Bryan So. An empirical study of the reliability of UNIX utilities. *Communications of the ACM*, 33(12):32–44, 1990. doi:10.1145/96267.96279.
- [26] MukulKolpe. Mukulkolpe/ethglobaltaipei. <https://github.com/MukulKolpe/ETHGlobalTaipei>, 2026. GitHub repository, accessed 2026.
- [27] Y. Oliinyk, M. Scott, R. Tsang, C. Fang, and H. Homayoun. Fuzzing BusyBox: Leveraging LLM and crash reuse for embedded bug unearthing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 883–900, 2024.
- [28] Omni Network. omni-network/omni. <https://github.com/omni-network/omni>, 2026. GitHub repository, accessed 2026.
- [29] Relay Protocol. relayprotocol/relay-depository. <https://github.com/relayprotocol/relay-depository>, 2026. GitHub repository, accessed 2026.
- [30] Michael Rodler, David Paaßen, Wenting Li, Lukas Bernhard, Thorsten Holz, Ghassan Karame, and Lucas Davi. Efcf: High performance smart contract fuzzing for exploit generation. In *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*, pages 449–471. IEEE, 2023. URL: <https://doi.org/10.1109/EuroSP57164.2023.00034>, doi:10.1109/EUROSP57164.2023.00034.
- [31] SarveshLimaye. Sarveshlimaye/trifecta. <https://github.com/SarveshLimaye/trifecta>, 2026. GitHub repository, accessed 2026.
- [32] Solady. vectorized/solady. <https://github.com/vectorized/solady>, 2026. GitHub repository, accessed 2026.
- [33] Christof Ferreira Torres, Antonio Ken Iannillo, Arthur Gervais, and Radu State. ConFuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In *Proceedings of the 2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 103–119, Vienna, Austria, 2021. doi:10.1109/EuroSP51992.2021.00018.
- [34] Trail of Bits. Medusa: Parallelized, coverage-guided, mutational Solidity smart contract fuzzing, powered by go-ethereum. <https://github.com/crytic/medusa>, 2024. Accessed 2026.
- [35] Pascal Winkler, Jens-Rene Giesen, Oussama Draissi, Federico Badaloni, Sebastian Holler, Clara Schneidewind, and Lucas Davi. \$2b lessons: Brigade as a defense against real-world defi bridge exploits. In Kangkook Jee and Aniket Kate, editors, *Applied Cryptography and Network Security*, pages 423–452, Cham, 2027. Springer Nature Switzerland.
- [36] Pascal Winkler, Christian Scholz, Jens-Rene Giesen, Noah Kappert, and Lucas Davi. Fuzzing cross-chain vulnerabilities with bridgefuzz. In *Proceedings of the 19th European Workshop on Systems Security, EuroSec 2026, Edinburgh, Scotland, UK, April 27-30, 2026*, pages 81–88. ACM, 2026. doi:10.1145/3803525.3804980.
- [37] Shuohan Wu, Zihao Li, Luyi Yan, Weimin Chen, Muhui Jiang, Chenxu Wang, Xiapu Luo, and Hao Zhou. Are we there yet? unraveling the state-of-the-art smart contract fuzzers. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*, pages 1–13, 2024.
- [38] Mengya Zhang, Xiaokuan Zhang, Josh Barbee, Yinqian Zhang, and Zhiqiang Lin. SoK: Security of cross-chain bridges: Attack surfaces, defenses, and open problems. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. ACM, 2024. arXiv:2312.12573. doi:10.1145/3678890.3678894.
- [39] Zequan Zhou, Xiling Luo, Xiaohai Ji, Jian Mao, Ting He, Junjun Wang, and Qianhong Wu. BridgeGuard: Checking external interaction vulnerabilities in cross-chain bridge router contracts based on symbolic dataflow analysis. *IEEE Transactions on Dependable and Secure Computing*, 22(5):5798–5812, 2025. doi:10.1109/TDSC.2025.3576114.